Group Report

Enhancing Image Recognition Accuracy via Optimization Algorithms

Ziheng LYU	2330036105
Zeming CHEN	2330036010
Jiaming LIU	2330036089
Xin WU	2330036159
Ang MA	2330036106
Songhao DENG	2330036015

Optimization Methods (1001)

Faculty of Business and Management

Tutor: Dr. Peimin CHEN

May 16

Contents

1 Project Overview and Objectives **1**
- 1.1 Project Introduction . . . 1
- 1.2 Project Goals . . . 1
- 1.3 Research Motivation . . . 1

2 PyTorch Implementation **2**
- 2.1 Core features . . . 2
- 2.2 Core Application . . . 2

3 CIFAR-10 Dataset **3**
- 3.1 Dataset Description . . . 3
- 3.2 Challenges for Tacking Dataset . . . 3

4 Theoretical Analysis of Optimization **4**
- 4.1 Optimization Algorithms . . . 4
  - 4.1.1 *Nesterov Momentum* . . . 4
  - 4.1.2 *RMSProp* . . . 5
  - 4.1.3 *Adam* . . . 6
  - 4.1.4 Algorithm Comparison . . . 6
- 4.2 Optimization Technique . . . 7
  - 4.2.1 Gradient Dropout . . . 7
  - 4.2.2 Gradient Noise . . . 8
  - 4.2.3 Method comparison . . . 8

5 Methodology: Network and Optimization **9**
- 5.1 Multilayer Perceptron (MLP) Architecture . . . 9
  - 5.1.1 Input Layer . . . 9
  - 5.1.2 First & Second Hidden Layer H_1 & H_2 . . . 9
  - 5.1.3 Output Layer . . . 10
- 5.2 Network Training: Backpropagation . . . 10
- 5.3 Optimization Algorithms for Parameter Updates . . . 11
  - 5.3.1 *SGD* and its Variants . . . 11
  - 5.3.2 *RMSProp* . . . 11
  - 5.3.3 *Adam* and its Variants . . . 12
  - 5.3.4 *L-BFGS* . . . 12

6 Experiment Process (code structure) 13
6.1 Code Structure 13
6.1.1 Networks.py 13
6.1.2 Optimizers.py 13
6.1.3 Misc.py 13
6.2 Experimental Workflow 14

7 Evaluation Indicators and Process 14
7.1 Evaluation Indicators 14
7.2 Evaluation Process 15

8 Result Analysis 15
8.1 Optimizers Comparison on Loss Convergence Performance 15
8.2 Optimizers Comparison on Accuracy, Efficiency, and Speed 16
8.3 Optimizers Recommendation 16
8.4 Future Work 17

A main.py 17

B misc.py 19

C networks.py 22

D optimizers.py 23

1 Project Overview and Objectives

1.1 Project Introduction

This project explores the role of optimization algorithms in the field of machine learning, with a particular focus on their application to neural network training.

Our study investigates both traditional and contemporary optimization methods.By conducting controlled experiments using standardized datasets and model architectures, we aim to highlight the strengths and trade-offs of different approaches.

1.2 Project Goals

The overarching goal of this project is to gain a deeper understanding of how optimization algorithms impact neural network training. More specifically, we aim to:

- **Implement** a selection of optimization algorithms—including both classical methods like SGD and advanced techniques such as Adam and RMSProp—within the PyTorch framework.
- **Compare** the effectiveness of these methods across multiple dimensions, including training speed, final test accuracy, robustness to initialization, and sensitivity to hyperparameters.
- **Analyze** convergence behavior through quantitative metrics and visualizations, such as loss curves and accuracy plots, in order to evaluate the efficiency and reliability of each optimizer.
- **Develop insights** into the practical considerations involved in selecting optimizers for real-world applications, with attention to computational cost and reproducibility.

Through this project, we hope to provide a clear and reproducible comparison of optimization strategies, which can serve as a practical reference for researchers and practitioners in the machine learning community.

1.3 Research Motivation

Optimizers play a critical role in the training process of deep learning models. Their importance stems from several key factors:

- **Training efficiency:** Optimizers significantly influence the speed at which a neural network converges during training, which directly affects development time.
- **Model performance:** The choice of optimizer impacts the final accuracy of the model and its ability to generalize to unseen data.
- **Resource usage:** Different optimization techniques vary in computational complexity and memory requirements, which is vital for real-world deployment scenarios.

This project addresses the challenge of identifying which optimization algorithms perform best under various training conditions.

2 PyTorch Implementation

During the optimization experiment, PyTorch, which is a powerful and open-source framework widely used in Computer Vision (CV) and Natural Language Processing (NLP). PyTorch is utilized to support Flexibly maintaining coding, running process, and debugging. Its internal advantages and conformity drive our optimization experiment mainly in acceleration of convergency speed and multi-file controlling system. Externally, PyTorch is compatible with neural networks project, advantaged in custom optimizers with direct inheritance from `torch.optim.Optimizer`, maintain code consistency by use main file to control separated file.

2.1 Core features

Three features including dynamic computational graphs, tensors with GPU acceleration and auto-grad system, form a recursive cycle to work cohesively.

- **Dynamic Computational Graphs**: Unlike static graph frameworks that pre-define operations, PyTorch builds the computational graph on-the-fly during execution, enabling real-time graph construction during forward pass, intuitive debugging using standard PyTorch tools, quick error detection as operation execute sequentially.
- **Tensors with GPU Acceleration**: PyTorch provide multi-dimensional array objects called Tensors which is similar to Numpy ndarray, enabling broadcasting, advanced indexing, matrix operations.
- **Auto-grad System**: This system which is related tensors can motivate automatic differentiation, enabling backward calculation through reverse-mode differentiation, Hessian matrix calculation, integration with computation graphs.

2.2 Core Application

All the applications of PyTorch is based on the integrated python file management system, which refers to using one main file to control and run other separated file, improving file organization and efficient code development, and easily debugging and environment prevention.

- **torch.nn Module**: Utilized for neural network layer construction, inserting implementations in “network.py”, connecting with MLP class.
- **torch.optim Module**: clarifying acquired optimization algorithms, extended in “optimizers.py” file.
- **Data loading**: using torch.utils.data module for the loading dataset.

- **GPU Acceleration**: Supported by Compute Unified Device Architecture (CUDA), improve training speed.

3 CIFAR-10 Dataset

The main object of the experiment is the CIFAR-10 dataset, which is based on the images, a foundational benchmark in computer vision, widely used for training and evaluating image classification models.

3.1 Dataset Description

CIFAR-10 contains 60,000 color images (32×32 pixels, see Figure 1) divided into 10 object categories: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, and truck, with 50000 images for training, 10000 images for testing. Each class has 6000 images, it originated from the "80 Million Tiny Images" dataset and was refined to address practical recognition challenges.



Figure 1: CIFAR-10 Sample

3.2 Challenges for Tacking Dataset

Due to the data structure of CIFAR-10, there are some challenges that may occur in the optimization process.

- **Low-Resolution Images and Spatial Feature Extraction**: Pixel Sparsity may occur in the process, indicates CIFAR-10 images lack fine-grained details, making it difficult for MLPs to capture spatial hierarchies or local patterns for instance, distinguishing bird feathers from textured backgrounds. High dimensionality

may increases overfitting risks, because MLPs treat pixels as independent features, leading to a large input layer (3,072 nodes) and parameter explosion in fully connected layers.

- **Intra-Class Variance and Ambiguity**: There are class overlap in similar images, confuses MLP's global feature aggregation. Moreover, color sensitivity may cause MLP overfit, ignoring shaped-based discriminative features.
- **Optimization and Regularization**: Vanishing gradients and unbalanced weight updates degrade convergence, especially in deeper MLP.

4 Theoretical Analysis of Optimization

4.1 Optimization Algorithms

4.1.1 *Nesterov Momentum*

The training cost of neural networks is very high. Its learning epoch requires a lot of time and money. The ordinary momentum method cannot adjust the direction adaptively and is prone to repeated oscillation near the bottom. Therefore, we hope that the constructed learning algorithm model can converge to a usable model within a tolerance interval with very few iterations. The first algorithm we considered was *Nesterov Momentum*

$$\mathbf{v}^{(k+1)} = \beta \mathbf{v}^{(k)} - \alpha \nabla f(\mathbf{x}^{(k)} + \beta \mathbf{v}^{(k)}) \tag{1}$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \mathbf{v}^{(k+1)} \tag{2}$$

where v is the momentum, α is the learning rate, β is the momentum decay, $\nabla f(x)$ is the gradient of function f given the variable x.

The reason we use it is that it is an improved momentum optimization algorithm that can solve the overshoot problem caused by momentum accumulation when the traditional momentum method is close to the minimum value. Its core idea is to adjust the parameter update direction by calculating the gradient of the "future position". Specifically, the algorithm introduces the momentum term β and the learning rate α. The momentum coefficient β controls the degree of retention of the historical gradient direction (usually 0.9), and the learning rate α determines the step size of each update. During the update process, the "future gradient" $\nabla f(\mathbf{x}^{(k)} + \beta \mathbf{v}^{(k)})$ is first calculated based on the sum of the current parameter position and the momentum term, and then the momentum is combined to correct the parameters.

This algorithm can not only help to overcome the convergence bottleneck period, but also reduce oscillations based on the predicted gradient, so as to approach the minimum value faster and more stably. Compared with traditional momentum or ordinary gradient descent, *Nesterov Momentum* performs better when the loss function surface is flat or there are local minima. The reason for choosing this algorithm is that it

takes into account both fast convergence and stability, especially for neural networks with high training computational costs and the need to avoid waste of computing power caused by repeated oscillations.

4.1.2 *RMSProp*

Later, we found that such an algorithm did not work well. The reason is that the fixed learning rate we used is not suitable for the model convergence of the entire stage. We found that the appropriate learning rate in the early stage of convergence is easy to be too small at the end of convergence. In other words, the appropriate learning rate at the end of training will cause the gradient to disappear during training. So we decided to use *RMSProp* as our improvement idea

$$\hat{\mathbf{s}}^{(k+1)} = \gamma \hat{\mathbf{s}}^{(k)} + (1-\gamma)\left(\mathbf{g}^{(k)} \odot \mathbf{g}^{(k)}\right) \tag{3}$$

$$x_i^{(k+1)} = x_i^{(k)} - \frac{\alpha}{\epsilon + \sqrt{\hat{s}_i^{(k)}}} g_i^{(k)} \tag{4}$$

where $\mathbf{s}^{(k)}$ is a vector whose i-th entry is the decayed sum of the squares of the partials, with respect to x_i, up to time step k; and ϵ is a small value, on the order of 1×10^{-8}, to prevent division by zero.

Because it is an adaptive learning rate optimization algorithm, it can solve the problem of monotonically decreasing learning rate of previous algorithms by dynamically adjusting the learning rate of each parameter. Its advantage is that it can adaptively balance the parameter update amplitude, which is especially suitable for neural networks with sparse data or uneven gradient distribution. Its core parameters include the decay rate γ (usually set to 0.9), the basic learning rate α and the numerical stability constant ϵ. When updating parameters, the learning rate is adjusted in the proportion of $\alpha/(\epsilon + \sqrt{\hat{\mathbf{s}}})$, so that the learning rate of frequently updated parameters is reduced and the learning rate of sparsely updated parameters is kept high.

It has been proven that *RMSProp* can solve the problem of large differences in the magnitude of gradients of different parameters in neural network parameter optimization, thereby avoiding the undesirable effects of manually adjusting the learning rate and fixing the learning rate. The reason for choosing *RMSProp* instead of Adagrad is that it avoids the problem of premature decay of the learning rate to zero through the decay mechanism. At the same time, compared with complex methods such as *Adam*, *RMSProp* has lower computational complexity and is suitable for scenarios that are sensitive to computing resources.

4.1.3 *Adam*

In the above two different approaches, we found that the advantages of the two algorithms should complement each other, so we decided to combine the momentum method with the adaptive learning rate mechanism and use the *Adam* algorithm as our most suitable optimization algorithm

$$\mathbf{v}^{(k+1)} = \gamma_v \mathbf{v}^{(k)} + (1 - \gamma_v)\, \mathbf{g}^{(k)} \tag{5}$$

$$\mathbf{s}^{(k+1)} = \gamma_s \mathbf{s}^{(k)} + (1 - \gamma_s) \left(\mathbf{g}^{(k)} \odot \mathbf{g}^{(k)} \right) \tag{6}$$

$$\hat{\mathbf{v}}^{(k+1)} = \mathbf{v}^{(k+1)} / \left(1 - \gamma_v^k \right) \tag{7}$$

$$\hat{\mathbf{s}}^{(k+1)} = \mathbf{s}^{(k+1)} / \left(1 - \gamma_s^k \right) \tag{8}$$

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} - \alpha \hat{\mathbf{v}}^{(k+1)} / \left(\epsilon + \sqrt{\hat{s}^{(k+1)}} \right) \tag{9}$$

It updates parameters by maintaining the first-order moment (gradient mean) and the second-order moment (gradient square mean). Its key parameters include momentum decay rate γ_v, second-order moment decay rate γ_s, learning rate α and stability constant ϵ. The algorithm first calculates the bias-corrected gradient mean $\hat{v}$ and square mean

The results of the *Adam* algorithm show the advantages of momentum acceleration equalization and dynamic adjustment of learning rate, while alleviating the estimation bias problem in the initial stage through bias correction. It is suitable for most of our preset examples, especially when the data distribution is complex or the parameter space is highly non-convex. The reason for choosing *Adam* over other algorithms (such as *SGD* or *Adagrad*) is that it is robust, has relatively simple hyperparameter adjustment, and can automatically adapt to the update requirements of different parameters. For example, in the task of back propagation, *Adam* usually converges faster than the pure momentum method and can handle noisy gradients better than *RMSProp*.

4.1.4 Algorithm Comparison

The common advantage of the above three algorithms (*Nesterov Momentum*, *RMSProp*, *Adam*) is to balance the convergence speed, stability and computational efficiency, but we also found that they also have advantages that other algorithms cannot match. *Nesterov Momentum* can converge quickly and shorten the number of epochs; *RMSProp* is suitable for handling sparse hyperparameter matrices and uneven parameter distribution; *Adam* is robust in most cases, especially in the case of neural network training tasks with extremely large hyperparameters. In contrast, traditional *SGD* is likely to fall into local minima and converges slowly, *Adagrad*'s learning rate decay mechanism

is too progressive, and algorithms such as *Adadelta* have high parameter adjustment complexity in practical applications. In addition, these algorithms all achieve the requirements of "low time complexity" and "avoiding overfitting" - *Nesterov Momentum* reduces oscillations through forward-looking gradients, and *RMSProp* and *Adam* suppress the influence of abnormal gradients through adaptive learning rates, thereby achieving efficient optimization with limited computing resources.

4.2 Optimization Technique

4.2.1 Gradient Dropout

When we train complex or deep networks, we find that the model tends to over-rely on specific features or noise in the training data (such as numbers, number of pixels, color, etc. in training), resulting in decreased generalization ability. The traditional Dropout method regularizes the results by randomly shielding the output of neurons, but cannot directly act on the gradient update process. Gradient Dropout randomly discards some gradient values during the learning process, forcing the model not to over-rely on some specific gradient directions when updating parameters, thereby more effectively suppressing overfitting.

Gradient Dropout can directly prevent the model from being overly sensitive to local noise in the training data by introducing a random deletion probability eta during the backpropagation process. By randomly shielding the gradient, the model is forced to explore more potential feature combinations when updating parameters, thereby improving generalization ability. Therefore, we put it in the residual network that requires fine control of parameter updates, so that it can directly act on the gradient flow rather than the intermediate activation value.

So we decided to make the following changes:

1. After back-propagation to obtain the gradient $\mathbf{g}$, generate a binary mask matrix $\mathbf{M}$ of the same dimension as the gradient, where each element M_i is 1 with probability p and 0 otherwise.
2. Multiply the gradient by the mask matrix element by element to get the gradient after discarding:
$$\mathbf{g}_{\text{drop}} = \mathbf{g} \odot \mathbf{M} \tag{10}$$
3. Use $\mathbf{g}_{\text{drop}}$ to update the parameters. The drop probability p is usually set to $[0.2, 0.5]$, depending on the needs of the example. Here we use an exhaustive method to pre-train the best parameters.

4.2.2 Gradient Noise

We also found that in the early stages of neural network training, the model may quickly converge to the local optimum due to the overly consistent gradient direction, but the generalization performance of such optimal solutions is poor. So we decided to switch the method by injecting noise instead of removing the gradient. The introduction of Gradient Noise adds random perturbations to the gradient by adding rather than removing them, preventing the optimization process from converging to sharp minima (that is, overfitting) too early. Gradient Noise makes the optimization path more exploratory, making it more likely to find a flat minimum area and improve the robustness of the model.

The main function of Gradient Noise is to simulate the implicit regularization effect in batch gradient descent (*SGD*) through the randomness of noise, while avoiding explicitly increasing the computational overhead. It can effectively prevent the model from being overly sensitive to small fluctuations in the training data and alleviate the problem of gradient vanishing or exploding. In addition, the introduction of noise enables the optimization process to maintain a certain degree of exploration ability when it is close to convergence, avoiding falling into suboptimal solutions. This mechanism is particularly prominent in training deep networks or adversarial sample generation tasks.

We implement Gradient Noise through the following steps:

1. After back-propagating the gradient $\mathbf{g}$, generate a noise vector $\boldsymbol{\epsilon}$ that follows a specific distribution,
2. Superimpose the noise on the original gradient:

$$\mathbf{g}_{\text{noise}} = \mathbf{g} + \boldsymbol{\epsilon} \tag{11}$$

   where $\boldsymbol{\epsilon}$ is followed by Normal Distribution $\mathcal{N}(0, \sigma^2)$ or uniform distribution $\text{Uniform}(0, 1)$.
3. Use $\mathbf{g}_{\text{noise}}$ to update the parameters. At the same time, we set the noise intensity σ to decay over time to reduce interference in the later stages of training.

$$\sigma = \frac{c}{(1+t)^{\gamma}} \tag{12}$$

   where t is the number of training steps.

4.2.3 Method comparison

Gradient Dropout and Gradient Noise both achieve regularization by interfering with the gradient update path, but the two algorithms focus on different aspects. Gradient Dropout forces the model to learn diversified features by structured gradient discarding, while Gradient Noise increases the randomness of the optimization process by

injecting noise. The two can be used in combination, especially when training deep networks, to prevent overfitting and improve the model's robustness to noise. Compared with traditional L_2 regularization

$$L_2 = w_1^2 + w_2^2 + \cdots + w_n^2 \tag{13}$$

or weight Dropout, both methods act directly on the gradient update mechanism and can more flexibly adapt to the needs of complex model structures.

5 Methodology: Network and Optimization

Previous section has illustrated the optimization algorithms. Therefore, this section intends to demonstrate the Multi-layer Perceptron (MLP) architecture employed in the project. Meanwhile, exploring backpropagation which is the vital part of network and demonstrating how optimization algorithms improve performance on classifying cifar-10 images by right of computed gradients are also main issues.

5.1 Multilayer Perceptron (MLP) Architecture

Our MLP architecture is built to classify image, specifically tailored here for datasets like CIFAR-10 after appropriate input flattening. This MLP is composed of an input layer, two hidden layers, and an output layer. Here are detailed explanation:

5.1.1 Input Layer

This layer takes flattened image. An image with C channels, height H, and width W results in the number of input features D_{in}:

$$D_{in} = C \times H \times W \tag{14}$$

For CIFAR-10 (if Din were adapted, e.g., 3072): $3 \times 32 \times 32 = 3072$. In our project, D_{in} is set to 3072 during instantiation in `main.py` when `args.dataset == cifar` and MLP is chosen.

5.1.2 First & Second Hidden Layer H_1 & H_2

This section demonstrates the hidden layer which learn the complex patterns from provided features from input layer.

1. **First Hidden Layer** H_1:
   N_{h1} neurons are contained where `networks.py` defaults $N_{h1} = 128$. Given an input vector $\mathbf{x} \in \mathbb{R}^{D_{in}}$, pre-activation $\mathbf{z}_{h1}$ and activation $\mathbf{a}_{h1}$ are:

$$\mathbf{z}_{h1} = \mathbf{W}_{h1}\mathbf{x} + \mathbf{b}_{h1} \tag{15}$$

$$\mathbf{a}_{h1} = \text{ReLU}\left(\mathbf{z}_{h1}\right) = \max\left(0, \mathbf{z}_{h1}\right) \tag{16}$$

where $\mathbf{W}_{h1} \in \mathbb{R}^{N_{h1} \times D_{in}}$ and $\mathbf{b}_{h1} \in \mathbb{R}^{N_{h1}}$.

2. **Second Hidden Layer** H_2: Contains N_{h2} neurons (default $N_{h2} = N_{h1}/2 = 64$). Then, input $\mathbf{a}_{h1}$:

$$\mathbf{z}_{h2} = \mathbf{W}_{h2}\mathbf{a}_{h1} + \mathbf{b}_{h2} \tag{17}$$

$$\mathbf{a}_{h2} = \text{ReLU}\left(\mathbf{z}_{h2}\right) \tag{18}$$

where $\mathbf{W}_{h2} \in \mathbb{R}^{N_{h2} \times N_{h1}}$ and $\mathbf{b}_{h2} \in \mathbb{R}^{N_{h2}}$.

5.1.3 Output Layer

Generates numerical scores for K classes ($K = 10$ in CIFAR-10 dataset). Input $\mathbf{a}_{h2}$:

$$\mathbf{z}_{\text{out}} = \mathbf{W}_{\text{out}}\,\mathbf{a}_{h2} + \mathbf{b}_{\text{out}} \tag{19}$$

where $\mathbf{W}_{\text{out}} \in \mathbb{R}^{K \times N_{h2}}$ and $\mathbf{b}_{\text{out}} \in \mathbb{R}^{K}$. With Softmax function, we can obtain the final probability $\hat{\mathbf{y}}$ of each class:

$$\hat{y}_k = \text{Softmax}\left(\mathbf{z}_{\text{out}}\right)_k = \frac{e^{z_{\text{out},k}}}{\sum_{j=1}^{K} e^{z_{\text{out},j}}} \tag{20}$$

We collectively denote the parameters of the network as $\theta = \{\mathbf{W}_{h1}, \mathbf{b}_{h1}, \mathbf{W}_{h2}, \mathbf{b}_{h2}, \mathbf{W}_{\text{out}}, \mathbf{b}_{\text{out}}\}$ and then can learn the complex pattern in the training phase.

5.2 Network Training: Backpropagation

With training a neural network, we minimize a loss function $L(\theta)$ that measures the discrepancy between the network's predictions $\hat{\mathbf{y}}$ and the true labels $\mathbf{y}$. In our classification task, Cross-Entropy Loss is utilized:

$$L = -\frac{1}{M}\sum_{i=1}^{M}\sum_{k=1}^{K} y_k^{(i)} \log\left(\hat{y}_k^{(i)}\right) \tag{21}$$

where M is the mini-batch size and $y_k^{(i)}$ is 1 if sample i belongs to class k, and 0 otherwise.

The Back-propagation algorithm is designed to compute the gradient of the loss function with respect to all network parameters $\nabla_\theta L$. For detailed calculation, it commonly applies chain rule of calculus iteratively where the algorithm starts from the output layer and moving backward through the network. Therefore, for any parameter $w \in \theta$:

$$\frac{\partial L}{\partial w} = \sum_{\text{paths}} \prod_{\text{edges in path}} (\text{local derivative}) \tag{22}$$

In this way, we can obtain the gradient for every weight and bias using $\nabla_{\mathbf{W}} L = \frac{\partial L}{\partial \mathbf{W}}$ and $\nabla_{\mathbf{b}} L = \frac{\partial L}{\partial \mathbf{b}}$. Hence, network can be updated.

5.3 Optimization Algorithms for Parameter Updates

With gradients $\mathbf{g}_t = \nabla_{\theta_t} L$ are computed at iteration t, optimizers is going to update the parameters θ_t to reach θ_{t+1} and η is the learning rate.

Basically, we utilize gradient descent to update the parameter θ_{t+1}. Nevertheless, there are some challenges to use it:

1. Sensitivity to learning rate η.
2. Can get stuck in local minima or saddle points.
3. Slow convergence in certain "ravine" landscapes.
4. Doesn't adapt step size per parameter.

Therefore, we intend to utilize previous introduced enhanced algorithms. In this section only involves a summary of each optimizers performs parameter updates in the training phase by right of **detailed introduction has been illustrated in previous section**.

5.3.1 *SGD* and its Variants

- ***SGD* with *Momentum***

$$\mathbf{v}_{t+1} = \mu \cdot \mathbf{v}_t - \eta \cdot \mathbf{g}_t \tag{23}$$

$$\theta_{t+1} = \theta_t + \mathbf{v}_{t+1} \tag{24}$$

  This algorithm introduces a velocity term $\mathbf{v}_t$ to speed up past gradients, accelerating convergency speed and oscillations.

- ***SGD* with *Nesterov***

$$\mathbf{v}_{t+1} = \mu \cdot \mathbf{v}_t - \eta \cdot \nabla_\theta L(\theta_t + \mu \cdot \mathbf{v}_t) \tag{25}$$

  This algorithm intend to reinforce momentum by virtue of calculating the gradient at a lookahead position, providing more accurate updates.

5.3.2 *RMSProp*

$$\mathbf{s}_t = \beta \cdot \mathbf{s}_{t-1} + (1 - \beta) \cdot \mathbf{g}_t^2 \tag{26}$$

$$\theta t + 1 = \theta_t - \frac{\eta}{\sqrt{\mathbf{s}_t + \epsilon}} \cdot \mathbf{g}_t \tag{27}$$

The algorithm utilizes adaptive learning rate for each parameter by right of historical average of squared gradient.

5.3.3 *Adam* and its Variants

- **Basic Adam**

$$\mathbf{m}_t = \beta_1 \cdot \mathbf{m}_{t-1} + (1 - \beta_1) \cdot \mathbf{g}_t \tag{28}$$

$$\mathbf{v}_t = \beta_2 \cdot \mathbf{v}_{t-1} + (1 - \beta_2) \cdot \mathbf{g}_t^2 \tag{29}$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t}, \quad \hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1 - \beta_2^t} \tag{30}$$

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \tag{31}$$

  This algorithm combines momentum and adaptive learning rates with bias correction.

- **AdamW**

$$\theta_{t+1} = \theta_t - \eta \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} - \lambda \cdot \theta_t \tag{32}$$

  This optimzer intends to decouples weight decay from the gradient descent update which can improve generalization

- **NAdam**

$$\mathbf{m}_t = \beta_1 \cdot \mathbf{m}_{t-1} + (1 - \beta_1) \cdot \mathbf{g}_{t-1} \tag{33}$$

$$\mathbf{v}_t = \beta_2 \cdot \mathbf{v}_{t-1} + (1 - \beta_2) \cdot \mathbf{g}_{t-1}^2 \tag{34}$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1 - \beta_1^t} \tag{35}$$

$$\theta_{t+1} = \theta_t - \eta \cdot \left(\frac{\beta_1 \cdot \hat{\mathbf{m}}_t + (1 - \beta_1) \cdot \mathbf{g}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \right) \tag{36}$$

  This optimzer expands adam by virtue of combining Nesterov momentum for lookahead gradient updates.

- **RAdam**

$$r_t = \sqrt{\frac{\left(1 - \beta_2^t\right)\left(\rho_t - 4\right)\left(\rho_t - 2\right)\rho_\infty}{\left(\rho_\infty - 4\right)\left(\rho_\infty - 2\right)\rho_t}} \tag{37}$$

$$\theta_{t+1} = \theta_t - \eta \cdot r_t \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\mathbf{v}_t} + \epsilon} \tag{38}$$

  This algorithm intends to stabilize adam in early training by right of rectifying variance with term r_t based on ρ_t.

5.3.4 *L-BFGS*

With numerous parameters in our network, it costs a lot in calculating using *L-BFGS* algorithm by right of storage of gradient history and parameter difference vector. Therefore, this part does not include the *L-BFGS* and is going to demonstrate the result in the following section.

6 Experiment Process (code structure)

The experimental workflow is designed to systematically compare the performance of various optimization algorithms (*SGD*, *Adam*, *RMSProp*, *L-BFGS*) for deep learning tasks on the CIFAR-10 dataset. This part primarily describes how they work by coding.

6.1 Code Structure

The experimental codes are organized into four primary modules: `Main.py`, `Networks.py`, `Optimizers.py`, and `Misc.py`. Each of them serves distinct functions to facilitate a structured and modular approach to the experiments.

6.1.1 Networks.py

`Networks.py` firstly defines a multilayer perceptron (MLP) with fully connected layers to flatten the input of CIFAR-10 images into a 3072-dimensional vector ($32 \times 32 \times 3$). Additionally, it defines the `fit()` function which encapsulates the training and testing loops with logging each epoch loss and accuracy, and finally returns the training and validation history for downstream analysis.

6.1.2 Optimizers.py

`Optimizers.py` implements a series of optimization algorithms, involving *SGD*, *Adam*, *RMSProp*, and *L-BFGS*. To enhance training stability and generalization, the module incorporates some advanced techniques:

- **Lookahead wrapper** to stabilize training
- **GradientNoise** to inject Gaussian noise for escaping local minima
- **GradientDropout** to randomly drop gradient components, thereby improving generalization.

6.1.3 Misc.py

`Misc.py` provides utility functions, including:

- **data loading** for CIFAR-10
- **optimizer configuration**
- **loss and accuracy tracking** via the `AvgMetric` class
- **saving metric histories** in `.pkl` file
- **generating comparative plots** for loss, accuracy
- **computational efficiency**

6.2 Experimental Workflow

The experimental workflow is executed by running the command-line `!python main.py -dataset cifar -num_epochs 30 -num_inits 5`.

1. **Initialization:** randomly select seed and choose suitable computational device (GPU).
2. **Load dataset:** using `misc.load_cifar` in `Misc.py` to load CIFAR-10 dataset and select subsets with 5000 training and 1000 validation samples.
3. **Optimizer Configuration:** using `misc.optim_dict` in `Misc.py` to retrieve optimizer configurations.
4. **Training Loop:** for each optimizer, perform 5 initializations with $3o$ epoch each, using `fit()` function in `Networks.py`. Also, computing the loss and accuracy per epoch.
5. **Result Aggregation:** aggregate metrics involve loss and accuracy across initializations, computing the mean and standard deviation using `misc.AvgMetric` and `aggregate_histories` from `Main.py`.
6. **Visualization:** generate loss and accuracy curves and save as PNG files by using `misc.plot_all_metrics`.

7 Evaluation Indicators and Process

7.1 Evaluation Indicators

There are three evaluation indicators involve in this experiment: training and test loss, training and test accuracy, and computational efficiency.

- **Training and test loss** are calculated using cross-entropy to quantify prediction deviation.
$$L = -\sum(y_i \times \log(\widetilde{y_i})), \tag{39}$$
where y_i is the true label and $\widetilde{y_i}$
- **Accuracy** is measured as the percentage of correctly classified samples.
$$Acc = \frac{\text{Number of Correct Prediction}}{\text{Total number of samples}} \tag{40}$$
- **Computational efficiency** is assessed based on the mean runtime of each optimizer, providing insights into their practical feasibility.

These indicators offer a comprehensive assessment for optimizer performance on the CIFAR-10 dataset.

7.2 Evaluation Process

The evaluation process utilizes the CIFAR-10 dataset, with 50,000 training and 10,000 test samples, and implement five independent runs per optimizer to account for their comprehensive performance.

1&2 **Train & Test:** Training is performed using the `fit()` function, while testing employs `torch.no_grad` to ensure fair and efficient outcomes.

3 **Log:** The `AvgMetric` class tracks each epoch statistics, including loss and accuracy.

4 **Aggregate:** Aggregated by calculating the mean and standard deviation across runs, Metrics are saved in `.pkl` format, and visualized as convergence curves.

5 **Compare:** Optimizers are ranked based on validation accuracy. Summary table is also generated to facilitate comparative analysis.

This structure ensures a fair and reproducible evaluation for optimizer performance.

8 Result Analysis

8.1 Optimizers Comparison on Loss Convergence Performance

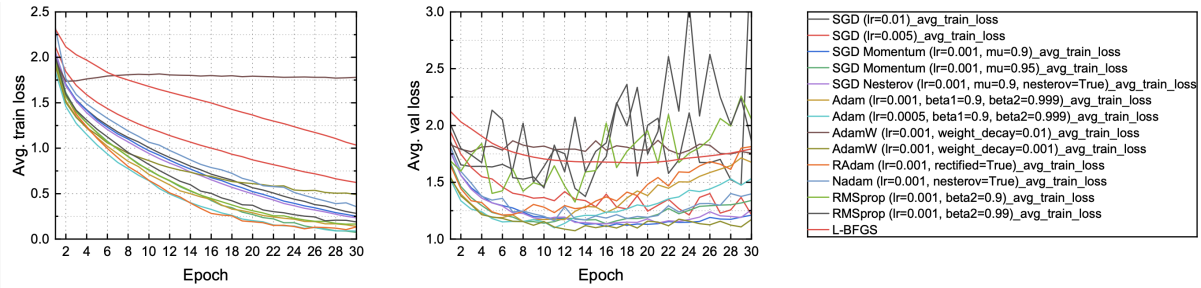


Figure 2: Performance of Algorithms

The Figure 2 illustrates the convergence behavior of various optimization algorithms over 30 training epochs, with average training loss shown on the left and average validation loss on the right. Several observations can be drawn regarding optimizer performance in terms of training efficiency, generalization, and stability.

SGD with learning rate 0.01 exhibits the worst performance in both training and validation, with high and unstable loss values, suggesting that the learning rate is too aggressive. In contrast, optimizers such as *AdamW*, *SGD Momentum* (with $\mu = 0.95$), and *RMSprop* demonstrate fast convergence during training and maintain low validation loss, indicating strong generalization capabilities. Notably, *L-BFGS* fails to reduce training loss effectively and yields poor validation performance, implying its incompatibility with the current network structure or data.

Furthermore, while all adaptive optimizers (Adam, AdamW, RMSprop, Nadam) achieve competitive training losses, their validation losses show varying degrees of

fluctuation. Among them, AdamW with weight decay = 0.001 achieves the most stable and lowest validation loss, highlighting the regularization benefit of weight decay in preventing overfitting.

8.2 Optimizers Comparison on Accuracy, Efficiency, and Speed

The table presents the final validation accuracy, validation loss, and average training time for a variety of optimization algorithms on the same training task. The results reveal significant differences in both model performance and computational efficiency.

SGD with momentum (0.001, 0.95) achieved the highest validation accuracy of 0.6276, while *Adam* (0.0005) followed closely at 0.6242. These two optimizers show strong generalization capability. In contrast, the plain *SGD* with learning rate 0.01 and *L-BFGS* produced the lowest accuracies (0.4118 and 0.4102, respectively), indicating instability or incompatibility with the model's loss surface.

From a loss perspective, *AdamW* (weight decay = 0.001) achieved the lowest validation loss of 1.1717, suggesting effective regularization. Both *SGD* (lr = 0.005) and *SGD* Momentum (0.001, μ = 0.9) also produced competitive low losses (1.21), while the standard *SGD* (lr = 0.01) had the highest loss (3.65), reflecting poor convergence.

Regarding computational efficiency, *L-BFGS* significantly outperformed other methods in terms of speed with an average training time of only 43.96 seconds, compared to 78 seconds for adaptive methods like *Adam*, *RMSprop*, and *Nadam*. However, its accuracy and loss results do not justify its use in this context.

8.3 Optimizers Recommendation

Based on the convergence curves and quantitative performance summary, *SGD* with *Momentum* (learning rate = 0.001, momentum = 0.95) and *AdamW* (learning rate = 0.001, weight decay = 0.001) emerge as the most recommended optimization algorithms in this experiment.

SGD Momentum demonstrates the best validation accuracy (0.6276) and a smooth, steadily declining loss curve, indicating strong training stability and generalization. Despite its simple formulation, the momentum term significantly accelerates convergence by preserving directionality across updates. Moreover, its relatively short average training time (73.57 s) makes it both effective and efficient.

AdamW, on the other hand, achieves the lowest validation loss (1.1717) and competitive accuracy (0.5952), while maintaining excellent loss stability across epochs. The inclusion of weight decay regularization helps prevent overfitting, which is evident from its smooth validation loss curve and generalization advantage over standard Adam or *RMSprop*.

In contrast, optimizers like *L-BFGS* and *SGD* with $lr = 0.01$ exhibit poor performance in both accuracy and loss, despite *L-BFGS* having the fastest training time.

Their instability and incompatibility with the model structure make them unsuitable for this task.

In conclusion, *SGD* Momentum is recommended for scenarios requiring high accuracy and training speed, while *AdamW* is ideal for tasks demanding strong generalization and regularization. Both optimizers offer a balanced trade-off between performance and efficiency.

8.4 Future Work

Building upon the current investigation, several avenues for future research can further enhance our understanding of optimization in deep learning. One promising direction is the exploration of more advanced optimizers or optimizer wrappers, such as *Lookahead* or *LAMB*, which may offer improvements in convergence stability and generalization. Additionally, applying these techniques to a broader range of neural network architectures and datasets could help evaluate their robustness and adaptability across different contexts. Another critical extension lies in hyperparameter optimization, where a more systematic study of learning rates, decay strategies, and momentum terms could uncover deeper insights into optimizer behavior. These extensions aim not only to refine our comparative framework but also to bring the findings closer to real-world deployment scenarios where efficiency, scalability, and reliability are paramount.

A main.py

```
import argparse
import torch
from torch.utils.data import DataLoader
import torch.nn as nn # For nn.CrossEntropyLoss
import numpy as np
import time
import os
import pickle as pkl

import misc # misc.py
import optimizers # optimizers.py
from networks import MLP, CNN, fit # networks.py

def run_experiment(args):
    """Main function to run experiments."""
    # --- Setup ---
    if args.seed is not None:
        torch.manual_seed(args.seed)
        np.random.seed(args.seed) # For numpy operations if any
        if torch.cuda.is_available():
            torch.cuda.manual_seed_all(args.seed)
            torch.backends.cudnn.deterministic = True # Can slow down, but good for reproducibility
            torch.backends.cudnn.benchmark = False   # Disable benchmarking for determinism

    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    print(f"Using device: {device}")

    # --- Data Loading ---
    dataset_seed = args.seed if args.seed is not None else 42 # Seed for data splitting
    if args.dataset == 'cifar':
        train_dataset, val_dataset = misc.load_cifar(num_train=args.num_train, num_val=args.num_val, seed=dataset_seed)
        NetModel = CNN # CNN class from networks.py
        criterion = nn.CrossEntropyLoss()
        # For CNN, input_channels might be needed if not fixed in CNN class
        # net_params = {'num_input_channels': 3, 'num_classes': 10} # Example
    elif args.dataset == 'mnist':
        train_dataset, val_dataset = misc.load_mnist(num_train=args.num_train, num_val=args.num_val, seed=dataset_seed)
        NetModel = MLP # MLP class from networks.py
        criterion = nn.CrossEntropyLoss()
        # net_params = {'num_features': 784, 'num_outputs': 10} # Example
    else:
        raise ValueError(f"Unknown dataset: {args.dataset}")

    train_loader = DataLoader(train_dataset, batch_size=args.batch_size, shuffle=True,
                              num_workers=args.num_workers, pin_memory=torch.cuda.is_available())
    val_loader = DataLoader(val_dataset, batch_size=args.batch_size, shuffle=False,
                            num_workers=args.num_workers, pin_memory=torch.cuda.is_available())
    print(f"Loaded {args.dataset.upper()} data: Train {len(train_dataset)}, Val {len(val_dataset)}")

    # --- Optimizer Configurations ---
```

```
all_optimizer_configs_to_test = misc.get_optimizer_configs()

if args.optimizer_key: # Filter if a specific optimizer base key is provided
    filtered_configs = [cfg for cfg in all_optimizer_configs_to_test if cfg['name'] == args.optimizer_key]
    if filtered_configs:
        all_optimizer_configs_to_test = filtered_configs
        print(f"Filtered to {len(all_optimizer_configs_to_test)} configurations for optimizer key '{args.optimizer_key}'.")
    else:
        print(f"Warning: Optimizer key '{args.optimizer_key}' not found. Running all {len(all_optimizer_configs_to_test)} configurations.")

# --- Results Storage ---
# This dictionary will store aggregated results (avg, std over inits) for each opt config
aggregated_results_for_all_configs = {}

# Path for the final aggregated results .pkl file
overall_results_dir = f'results_{args.dataset}/'
os.makedirs(overall_results_dir, exist_ok=True)
aggregated_pkl_filepath = os.path.join(overall_results_dir, f'all_optimizer_results_aggregated_{args.dataset}.pkl')

# --- Main Experiment Loop ---
if not args.only_plot:
    for opt_config_dict in all_optimizer_configs_to_test:
        detailed_opt_label = opt_config_dict['label']
        print(f"\n===== Running Experiment for: {detailed_opt_label} =====")

        # Store metrics from each initialization run for THIS optimizer config
        current_config_train_loss_histories = []
        current_config_val_loss_histories = []
        current_config_train_acc_histories = []
        current_config_val_acc_histories = []
        current_config_total_times_per_init = []

        for i_init in range(args.num_inits):
            print(f"--- Initialization {i_init+1}/{args.num_inits} for {detailed_opt_label} ---")

            # Set seed for this specific initialization run for model weights
            # Data splitting seed is handled once above.
            current_init_seed = (args.seed if args.seed is not None else np.random.randint(0, 10000)) + i_init
            torch.manual_seed(current_init_seed)
            if torch.cuda.is_available(): torch.cuda.manual_seed_all(current_init_seed)

            # Initialize Model (model __init__ moves it to device)
            # Pass dataset-specific params if your NetModel class requires them
            model_instance = NetModel() # e.g., NetModel(**net_params) if needed

            # Optimizer Instantiation
            opt_class = opt_config_dict['optimizer_class']
            opt_kwargs_for_class = opt_config_dict['kwargs']
            optimizer = opt_class(model_instance.parameters(), **opt_kwargs_for_class)

            # Optimizer Wrapping (if any)
            wrapper_type = opt_config_dict.get('wrapper_type')
            if wrapper_type:
                wrapper_specific_kwargs = opt_config_dict.get('wrapper_kwargs', {})
                optimizer = misc.wrap_optimizer(optimizer, wrapper_type, wrapper_specific_kwargs,
                                                task_name_for_logic=opt_config_dict.get('task_name_for_wrapper_logic'))

            # Learning Rate Scheduler
            lr_sched = None
            if args.lr_schedule_type == 'ReduceLROnPlateau':
                lr_sched = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1,
                                                                      patience=args.lr_patience, verbose=False)
            elif args.lr_schedule_type == 'StepLR':
                lr_sched = torch.optim.lr_scheduler.StepLR(optimizer, step_size=args.lr_step_size, gamma=0.1)

            # --- Training & Validation for this init run ---
            run_start_time = time.time()
            train_loss_hist, val_loss_hist, train_acc_hist, val_acc_hist = fit(
                net_model=model_instance,
                train_data_loader=train_loader,
                val_data_loader=val_loader,
                optimizer_instance=optimizer,
                criterion=criterion,
                num_epochs_to_run=args.num_epochs,
                device_to_use=device,
                lr_scheduler=lr_sched,
                epoch_print_interval=args.verbose_interval
            )
            run_elapsed_time = time.time() - run_start_time
            print(f"Initialization {i_init+1} finished in {run_elapsed_time:.2f}s.")

            # Save raw history for this init run (optional, but good for deep dives)
            # misc.save_metric_history(train_loss_hist, args.dataset, detailed_opt_label, 'train_loss_raw', i_init)
            # ... (save other raw histories if needed) ...

            current_config_train_loss_histories.append(train_loss_hist)
            current_config_val_loss_histories.append(val_loss_hist)
            current_config_train_acc_histories.append(train_acc_hist)
            current_config_val_acc_histories.append(val_acc_hist)
            current_config_total_times_per_init.append(run_elapsed_time)

        # --- Aggregate results for the current optimizer configuration ---
        if current_config_train_loss_histories: # If any inits were successful
            # Helper to average/std over lists of lists (histories from inits)
            # Ensures all inner lists are same length (num_epochs) for np operations
            def aggregate_histories(list_of_histories, num_epochs_expected):
                valid_histories = [h for h in list_of_histories if h is not None and len(h) == num_epochs_expected]
                if not valid_histories: # No valid histories to aggregate
                    return np.array([]), np.array([])
                try:
                    np_array_of_histories = np.array(valid_histories)
                    return np.mean(np_array_of_histories, axis=0), np.std(np_array_of_histories, axis=0)
                except Exception as e:
                    print(f"Error during history aggregation: {e}. Histories: {list_of_histories}")
                    return np.array([]), np.array([])


            avg_train_loss, std_train_loss = aggregate_histories(current_config_train_loss_histories, args.num_epochs)
            avg_val_loss, std_val_loss = aggregate_histories(current_config_val_loss_histories, args.num_epochs)
            avg_train_acc, std_train_acc = aggregate_histories(current_config_train_acc_histories, args.num_epochs)
            avg_val_acc, std_val_acc = aggregate_histories(current_config_val_acc_histories, args.num_epochs)

            aggregated_results_for_all_configs[detailed_opt_label] = {
                'avg_train_loss': avg_train_loss.tolist() if avg_train_loss.size > 0 else [],
                'std_train_loss': std_train_loss.tolist() if std_train_loss.size > 0 else [],
                'avg_val_loss': avg_val_loss.tolist() if avg_val_loss.size > 0 else [],
```

```
                    'std_val_loss': std_val_loss.tolist() if std_val_loss.size > 0 else [],
                    'avg_train_acc': avg_train_acc.tolist() if avg_train_acc.size > 0 else [],
                    'std_train_acc': std_train_acc.tolist() if std_train_acc.size > 0 else [],
                    'avg_val_acc': avg_val_acc.tolist() if avg_val_acc.size > 0 else [],
                    'std_val_acc': std_val_acc.tolist() if std_val_acc.size > 0 else [],
                    'avg_total_time': np.mean(current_config_total_times_per_init) if current_config_total_times_per_init else 0,
                    'std_total_time': np.std(current_config_total_times_per_init) if len(current_config_total_times_per_init) > 1 else 0,
                    'num_successful_inits': len([h for h in current_config_train_loss_histories if h is not None and len(h) == args.num_epochs])
                }
                print(f"Aggregated results for {detailed_opt_label} over {aggregated_results_for_all_configs[detailed_opt_label]['num_successful_inits']} inits.")

        # Save the dictionary containing all aggregated results
        if aggregated_results_for_all_configs:
            with open(aggregated_pkl_filepath, 'wb') as f:
                pkl.dump(aggregated_results_for_all_configs, f)
            print(f"\nSaved all aggregated results to: {aggregated_pkl_filepath}")
        else:
            print("\nNo training was performed or no results to save.")

    # --- Plotting and Summary ---
    if args.only_plot or aggregated_results_for_all_configs: # Plot if we just trained or if in only_plot mode
        if args.only_plot: # If only_plot, try to load the aggregated results
            try:
                with open(aggregated_pkl_filepath, 'rb') as f:
                    aggregated_results_for_all_configs = pkl.load(f)
                print(f"Loaded aggregated results from: {aggregated_pkl_filepath}")
            except FileNotFoundError:
                print(f"Error: Aggregated results file '{aggregated_pkl_filepath}' not found for plotting. Run training first.")
                return
            except Exception as e:
                print(f"Error loading aggregated results file: {e}")
                return

        if not aggregated_results_for_all_configs:
            print("No results available to plot or summarize.")
            return

        print("\n----- Generating Plots -----")
        misc.plot_all_metrics(
            aggregated_results_data_dict=aggregated_results_for_all_configs,
            num_epochs_completed=args.num_epochs, # All runs are for this many epochs
            dataset_name_str=args.dataset,
            plot_std_dev_bands=True # Enable error bands
        )

        print("\n----- Performance & Efficiency Summary -----")
        # Sort by final average validation accuracy (descending)
        # Handle cases where 'avg_val_acc' might be empty or not present for some configs
        def get_final_val_acc(metrics_dict):
            val_acc_hist = metrics_dict.get('avg_val_acc')
            return val_acc_hist[-1] if val_acc_hist and len(val_acc_hist) > 0 else -1.0 # Default to -1 if no acc data

        sorted_configs_by_perf = sorted(aggregated_results_for_all_configs.items(),
                                        key=lambda item: get_final_val_acc(item[1]),
                                        reverse=True)

        # Header for the summary table
        header = f"{'Optimizer Configuration':<70} | {'Final Val Acc':<18} | {'Final Val Loss':<18} | {'Avg Train Time (s)':<20} | {'Inits OK':<8}"
        print(header)
        print("-" * len(header))

        for opt_label, metrics in sorted_configs_by_perf:
            final_val_acc_str = "N/A"
            if metrics.get('avg_val_acc') and metrics['avg_val_acc']:
                final_val_acc_str = f"{metrics['avg_val_acc'][-1]:.4f} \u00B1 {metrics['std_val_acc'][-1]:.4f}"

            final_val_loss_str = "N/A"
            if metrics.get('avg_val_loss') and metrics['avg_val_loss']:
                final_val_loss_str = f"{metrics['avg_val_loss'][-1]:.4f} \u00B1 {metrics['std_val_loss'][-1]:.4f}"

            avg_time_str = f"{metrics.get('avg_total_time', 0):.2f} \u00B1 {metrics.get('std_total_time', 0):.2f}"
            inits_ok_str = str(metrics.get('num_successful_inits',0))

            print(f"{opt_label:<70} | {final_val_acc_str:<18} | {final_val_loss_str:<18} | {avg_time_str:<20} | {inits_ok_str:<8}")
    else:
         print("No results to plot (either training was skipped and no pkl file found, or no results were generated).")


if __name__ == '__main__':
    parser = argparse.ArgumentParser(description="Deep Learning Optimizer Comparison Framework")

    # Experiment Setup
    parser.add_argument('-dataset', type=str, default='cifar', choices=['cifar', 'mnist'], help="Dataset to use")
    parser.add_argument('-num_epochs', type=int, default=20, help="Number of training epochs per initialization") # Reduced for faster runs
    parser.add_argument('-num_inits', type=int, default=2, help="Number of random initializations per optimizer config") # Default 2 for speed
    parser.add_argument('-seed', type=int, default=None, help="Global random seed for reproducibility (for data splits, init seeds). If None, non-deterministic.")

    # Data Handling
    parser.add_argument('-num_train', type=int, default=5000, help="Number of training samples (subset)")
    parser.add_argument('-num_val', type=int, default=1000, help="Number of validation samples (subset)")
    parser.add_argument('-batch_size', type=int, default=64, help="Batch size for training and validation") # Smaller batch for potentially smaller datasets
    parser.add_argument('-num_workers', type=int, default=2, help="Dataloader workers. Set to 0 if issues in Colab/Windows.")

    # LR Scheduling
    parser.add_argument('-lr_schedule_type', type=str, default=None, choices=['ReduceLROnPlateau', 'StepLR'], help="Type of LR scheduler")
    parser.add_argument('-lr_patience', type=int, default=3, help="Patience for ReduceLROnPlateau (epochs)") # Shorter patience
    parser.add_argument('-lr_step_size', type=int, default=7, help="Step size for StepLR (epochs)") # Adjusted step size

    # Filtering and Mode
    parser.add_argument('-optimizer_key', type=str, default=None, help="Run only a specific optimizer base key from optim_dict (e.g., 'adam', 'sgd')")
    parser.add_argument('-only_plot', action='store_true', help="Skip training, load aggregated results and plot")

    # Logging
    parser.add_argument('-verbose_interval', type=int, default=1, help="Print training progress every N epochs")

    cli_args = parser.parse_args()
    run_experiment(cli_args)
```

B misc.py

```
import os
```

```
import pickle as pkl
from copy import deepcopy

import matplotlib.pyplot as plt
import numpy as np
from scipy.signal import savgol_filter
import seaborn as sns
import torch
import torchvision
from torch.utils import data
from torchvision import transforms

import optimizers

optim_dict = {
  'sgd': {
   'label': 'SGD',
   'lr': 1e-3
  },
  'sgd_momentum': {
   'label': 'SGD w/ momentum',
   'lr': 1e-3,
   'mu': 0.99
  },
  'sgd_nesterov': {
   'label': 'SGD w/ Nesterov momentum',
   'lr': 1e-3,
   'mu': 0.99,
   'nesterov': True
  },
  'sgd_weight_decay': {
   'label': 'SGDW',
   'lr': 1e-3,
   'mu': 0.99,
   'weight_decay': 1e-6
  },
  'sgd_lrd': {
   'label': 'SGD w/ momentum + LRD',
   'lr': 1e-3,
   'mu': 0.99,
   'lrd': 0.5
  },
  'adam': {
   'label': 'Adam',
   'lr': 1e-3
  },
  'adamW':{
   'label': 'AdamW',
   'lr': 1e-3,
   'weight_decay': 1e-4
  },
  'adam_l2':{
   'label': 'AdamL2',
   'lr': 1e-3,
   'l2_reg': 1e-4
  },
  'adam_lrd': {
   'label': 'Adam w/ LRD',
   'lr': 1e-3,
   'lrd': 0.5
  },
  'Radam': {
   'label': 'RAdam',
   'lr': 1e-3,
   'rectified': True
  },
  'RadamW': {
   'label': 'RAdamW',
   'lr': 1e-3,
   'rectified': True,
   'weight_decay': 1e-4
  },
  'Radam_lrd': {
   'label': 'RAdam w/ LRD',
   'lr': 1e-3,
   'rectified': True,
   'lrd': 0.5
  },
  'nadam': {
   'label': 'Nadam',
   'lr': 1e-3,
   'nesterov': True
  },
  'rmsprop': {
   'label': 'RMSprop',
   'lr': 1e-3,
   'beta2': 0.9,
  },
  'lookahead_sgd': {
   'label': 'Lookahead (SGD)',
   'lr': 1e-3,
   'mu': 0.99
  },
  'lookahead_adam': {
   'label': 'Lookahead (Adam)',
   'lr': 1e-3
  },
  'gradnoise_adam': {
   'label': 'Gradient Noise (Adam)',
   'lr': 1e-3
  },
  'graddropout_adam': {
   'label': 'Gradient Dropout (Adam)',
   'lr': 1e-3
  }
 }


def split_optim_dict(d:dict) -> tuple:
 """
 Splits an optimization dict into label and dict.
 """
 temp_d = deepcopy(d)
 label = temp_d['label']
 del temp_d['label']
```

```
 return label, temp_d


def load_cifar(num_train=50000, num_val=2048):
 """
 Loads a subset of the CIFAR dataset and returns it as a tuple.
 """
 transform = transforms.Compose([transforms.ToTensor(), transforms.Normalize((0.5, 0.5, 0.5),(0.5, 0.5, 0.5))])

 train_dataset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
 val_dataset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)

 train_dataset, _ = torch.utils.data.random_split(train_dataset, lengths=[num_train, len(train_dataset)-num_train])
 val_dataset, _ = torch.utils.data.random_split(val_dataset, lengths=[num_val, len(val_dataset)-num_val])

 return train_dataset, val_dataset


def load_mnist(filename='data/mnist.npz', num_train=4096, num_val=512):
 """
 Loads a subset of the grayscale MNIST dataset and returns it as a tuple.
 """
 data = np.load(filename)

 x_train = data['X_train'][:num_train].astype('float32')
 y_train = data['y_train'][:num_train].astype('int32')

 x_valid = data['X_valid'][:num_val].astype('float32')
 y_valid = data['y_valid'][:num_val].astype('int32')

 train_dataset = Dataset(x_train, y_train)
 val_dataset = Dataset(x_valid, y_valid)

 return train_dataset, val_dataset


def task_to_optimizer(task:str) -> torch.optim.Optimizer:
 """
 Takes a task as string and returns its respective optimizer class.
 """
 optimizer = None

 if 'sgd' in task.lower():
  optimizer = getattr(optimizers, 'SGD')
 if 'adam' in task.lower():
  optimizer = getattr(optimizers, 'Adam')
 if 'rmsprop' in task.lower():
  optimizer = getattr(optimizers, 'RMSProp')

 if optimizer is None:
  raise ValueError(f'Optimizer for task \'{task}\' was not recognized!')

 return optimizer


def wrap_optimizer(task:str, optimizer):
 """
 Wraps an instantiated optimizer according to its task specified as a string.
 """
 if 'gradnoise' in task.lower():
  optimizer = optimizers.GradientNoise(optimizer, eta=0.3, gamma=0.55)

 if 'graddropout' in task.lower():
  optimizer = optimizers.GradientDropout(optimizer, grad_retain=0.9)

 if 'lookahead' in task.lower():
  optimizer = optimizers.Lookahead(optimizer, k=5, alpha=0.5)

 return optimizer


class AvgLoss():
 """
 Utility class that tracks the average loss.
 """
 def __init__(self):
  self.sum, self.avg, self.n = 0, 0, 0
  self.losses = []

 def __iadd__(self, other):
  try:
   loss = other.data.numpy()
  except:
   loss = other

  if isinstance(other, list):
   self.losses.extend(other)
   self.sum += np.sum(other)
   self.n += len(other)
  else:
   self.losses.append(float(loss))
   self.sum += loss
   self.n += 1

  self.avg = self.sum / self.n

  return self

 def __str__(self):
  return '{0:.4f}'.format(round(self.avg, 4))

 def __len__(self):
  return len(self.losses)


class Dataset(data.Dataset):
 def __init__(self, X, y):
  self.X = X
  self.y = y

 def __len__(self):
  return len(self.X)

 def __getitem__(self, idx):
  return self.X[idx], self.y[idx]
```

```
def save_losses(losses, dataset:str, filename:str):
 if not os.path.exists(f'losses_{dataset}/'): os.makedirs(f'losses_{dataset}/')
 with open(f'losses_{dataset}/{filename}.pkl', 'wb') as f:
  pkl.dump(losses, f, protocol=pkl.HIGHEST_PROTOCOL)


def load_losses(dataset:str, filename:str):
 try:
  with open(f'losses_{dataset}/{filename}.pkl', 'rb') as f:
   return pkl.load(f)
 except:
  return None


def plot_mnist(X):
 idx, dim, classes = 0, 28, 10
 canvas = np.zeros((dim*classes, classes*dim))

 for i in range(classes):
  for j in range(classes):
   canvas[i*dim:(i+1)*dim, j*dim:(j+1)*dim] = X[idx].reshape((dim, dim))
   idx += 1

 sns.set(style='darkgrid')
 plt.figure(figsize=(9, 9))
 plt.axis('off')
 plt.tight_layout(pad=0)
 plt.imshow(canvas, cmap='gray')
 plt.savefig('mnist_examples.png')
 plt.clf()


def plot_loss(losses, val_losses, num_epochs):
 sns.set(style='darkgrid')
 plt.figure(figsize=(12, 6))
 plt.plot(np.linspace(0, num_epochs, num=len(losses)), losses.losses, label='Training loss')
 plt.plot(np.linspace(0, num_epochs, num=len(val_losses)), val_losses.losses, label='Validation loss')
 plt.tight_layout(pad=2)
 plt.xlabel('Epoch')
 plt.ylabel('Negative log likelihood')
 plt.savefig('loss.png')
 plt.clf()


def plot_losses(losses, val_losses, labels, num_epochs, title, plot_val=False, yscale_log=False, max_epochs=None):
 sns.set(style='darkgrid')
 plt.figure(figsize=(12, 6))
 colors = ['tab:blue', 'tab:orange', 'tab:green', 'tab:red', 'tab:purple', 'tab:brown', 'tab:pink', 'tab:gray', 'tab:cyan', 'tab:olive']

 for i in range(len(losses)):
  plt.plot(np.linspace(0, num_epochs, num=len(losses[i])), smooth(losses[i].losses, 81), label=labels[i], alpha=1, c=colors[i])
  plt.plot(np.linspace(0, num_epochs, num=len(losses[i])), smooth(losses[i].losses, 21), alpha=0.25, c=colors[i])
  if plot_val:
   plt.plot(np.linspace(0, num_epochs, num=len(val_losses[i])), smooth(val_losses[i].losses, 81), alpha=1, linestyle='--', c=colors[i])

 plt.tight_layout(pad=2)
 plt.xlabel('Epoch')
 plt.ylabel('Cross-entropy')
 if yscale_log:
  plt.yscale('log')
 if max_epochs is not None:
  plt.xlim(-1, max_epochs)
 plt.ylim(0, 3)
 plt.title('CNN benchmark on CIFAR-10' if title == 'cifar' else 'MLP benchmark on MNIST')
 plt.legend(loc='upper right')
 plt.savefig(f'loss_{title}.png')
 plt.clf()


def smooth(signal, kernel_size, polyorder=3):
 return savgol_filter(signal, kernel_size, polyorder)
```

C networks.py

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.nn.init as init
from torch import utils
from torch.nn import Parameter

import misc


class MLP(nn.Module):
 """
 A small multilayer perceptron with parameters that we can optimize for the task.
 """
 def __init__(self, num_features=784, num_hidden=64, num_outputs=10):
  super(MLP, self).__init__()

  self.W_1 = Parameter(init.xavier_normal_(torch.Tensor(num_hidden, num_features)))
  self.b_1 = Parameter(init.constant_(torch.Tensor(num_hidden), 0))

  self.W_2 = Parameter(init.xavier_normal_(torch.Tensor(num_outputs, num_hidden)))
  self.b_2 = Parameter(init.constant_(torch.Tensor(num_outputs), 0))

 def forward(self, x):
  x = F.relu(F.linear(x, self.W_1, self.b_1))
  x = F.linear(x, self.W_2, self.b_2)

  return x


class CNN(nn.Module):
 """
 A small convolutional neural network with parameters that we can optimize for the task.
 """
 def __init__(self, num_layers=4, num_filters=64, num_classes=10, input_size=(3, 32, 32)):
  super(CNN, self).__init__()
```

```
  self.channels = input_size[0]
  self.height = input_size[1]
  self.width = input_size[2]
  self.num_filters = num_filters

  self.conv_in = nn.Conv2d(self.channels, self.num_filters, kernel_size=5, padding=2)
  cnn = []
  for _ in range(num_layers):
   cnn.append(nn.Conv2d(self.num_filters, self.num_filters, kernel_size=3, padding=1))
   cnn.append(nn.BatchNorm2d(self.num_filters))
   cnn.append(nn.ReLU())
  self.cnn = nn.Sequential(*cnn)

  self.out_lin = nn.Linear(self.num_filters*self.width*self.height, num_classes)

  if torch.cuda.is_available():
   self.cuda()


 def forward(self, x):
  if torch.cuda.is_available():
   x = x.cuda()

  x = F.relu(self.conv_in(x))
  x = self.cnn(x)
  x = x.reshape(x.size(0), -1)

  return self.out_lin(x)


def fit(net, data, optimizer, batch_size=128, num_epochs=250, lr_schedule=False):
 """
 Fits parameters of a network ‘net‘ using ‘data‘ as training data and a given ‘optimizer‘.
 """
 train_generator = utils.data.DataLoader(data[0], batch_size=batch_size)
 val_generator = utils.data.DataLoader(data[1], batch_size=batch_size)

 losses = misc.AvgLoss()
 val_losses = misc.AvgLoss()

 if lr_schedule:
  scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer, patience=5)

 for epoch in range(num_epochs+1):

  epoch_loss = misc.AvgLoss()
  epoch_val_loss = misc.AvgLoss()

  for x, y in val_generator:
   y = y.type(torch.LongTensor)
   if torch.cuda.is_available(): y = y.cuda()
   epoch_val_loss += F.cross_entropy(net(x), y).cpu()

  for x, y in train_generator:
   y = y.type(torch.LongTensor)
   if torch.cuda.is_available(): y = y.cuda()
   loss = F.cross_entropy(net(x), y).cpu()
   epoch_loss += loss

   optimizer.zero_grad()
   loss.backward()
   optimizer.step()

  if lr_schedule:
   scheduler.step(epoch_loss.avg)

  if epoch % 2 == 0:
   print(f'Epoch {epoch}/{num_epochs}, loss: {epoch_loss}, val loss: {epoch_val_loss}')

  losses += epoch_loss.losses
  val_losses += epoch_val_loss.losses

 return losses, val_losses
```

D optimizers.py

```
import torch
import numpy as np
from torch.optim import Optimizer
from torch.distributions import Bernoulli, Normal


class SGD(Optimizer):
    """
    Stochastic gradient descent. Also includes implementations of momentum,
    Nesterov's momentum, L2 regularization, SGDW and Learning Rate Dropout.
    """
    def __init__(self, params, lr, mu=0, nesterov=False, weight_decay=0, lrd=1):
        defaults = {'lr': lr, 'mu': mu, 'nesterov': nesterov, 'weight_decay': weight_decay, 'lrd': lrd}
        super(SGD, self).__init__(params, defaults)

    def step(self):
        """
        Performs a single optimization step.
        """
        for group in self.param_groups:

            lr = group['lr']
            mu = group['mu']
            nesterov = group['nesterov']
            weight_decay = group['weight_decay']
            lrd_bernoulli = Bernoulli(probs=group['lrd'])

            if mu != 0 and 'v' not in group:
                group['v'] = []
                if nesterov:
                    group['theta'] = []
                for param in group['params']:
                    group['v'].append(torch.zeros_like(param))
                    if nesterov:
```

```
                        theta_param = torch.ones_like(param).mul_(param.data)
                        group['theta'].append(theta_param)

            for idx, param in enumerate(group['params']):
                param.grad.data -= weight_decay * param.data
                lrd_mask = lrd_bernoulli.sample(param.size()).to(param.device)

                if mu != 0:
                    v = group['v'][idx]
                    v = mu * v - lr * param.grad.data
                    group['v'][idx] = v

                    if nesterov:
                        group['theta'][idx] += lrd_mask * v
                        param.data = group['theta'][idx] + mu * v

                    else:
                        param.data += lrd_mask * v

                else:
                    param.data -= lrd_mask * lr * param.grad.data


class Adam(Optimizer):
    """
    Adam as proposed by https://arxiv.org/abs/1412.6980.
    Also includes a number of proposed extensions to the the Adam algorithm,
    such as Nadam, L2 regularization, AdamW, RAdam and Learning Rate Dropout.
    """
    def __init__(self, params, lr, beta1=0.9, beta2=0.999, nesterov=False, l2_reg=0, weight_decay=0, rectified=False, lrd=1, eps=1e-8):
        defaults = {'lr': lr, 'beta1': beta1, 'beta2': beta2, 'nesterov': nesterov, 'l2_reg': l2_reg,
                    'weight_decay': weight_decay, 'rectified': rectified, 'lrd': lrd, 'eps': eps}
        super(Adam, self).__init__(params, defaults)

    def step(self):
        """
        Performs a single optimization step.
        """
        for group in self.param_groups:

            lr = group['lr']
            beta1 = group['beta1']
            beta2 = group['beta2']
            nesterov = group['nesterov']
            l2_reg = group['l2_reg']
            weight_decay = group['weight_decay']
            rectified = group['rectified']
            lrd_bernoulli = Bernoulli(probs=group['lrd'])
            eps = group['eps']

            if 'm' not in group and 'v' not in group:
                group['m'] = []
                group['v'] = []
                group['t'] = 1
                if nesterov:
                    group['prev_grad'] = []
                for param in group['params']:
                    group['m'].append(torch.zeros_like(param))
                    group['v'].append(torch.zeros_like(param))
                    if nesterov:
                        group['prev_grad'].append(torch.zeros_like(param))

            for idx, param in enumerate(group['params']):
                if l2_reg:
                    param.grad.data += l2_reg * param.data

                if nesterov:
                    grad = group['prev_grad'][idx]
                else:
                    grad = param.grad.data

                lrd_mask = lrd_bernoulli.sample(param.size()).to(param.device)

                m = group['m'][idx]
                v = group['v'][idx]
                t = group['t']
                m = beta1 * m + (1 - beta1) * grad
                v = beta2 * v + (1 - beta2) * grad**2
                m_hat = m / (1 - beta1**t)
                v_hat = v / (1 - beta2**t)

                if nesterov:
                    group['prev_grad'][idx] = param.grad.data

                if rectified:
                    rho_inf = 2 / (1 - beta2) - 1
                    rho = rho_inf - 2 * t * beta2**t / (1 - beta2**t)
                    if rho >= 5:
                        numerator = (1 - beta2**t) * (rho - 4) * (rho - 2) * rho_inf
                        denominator = (rho_inf - 4) * (rho_inf - 2) * rho
                        r = np.sqrt(numerator / denominator)
                        param.data += - lrd_mask * lr * r * m_hat / (torch.sqrt(v) + eps)
                    else:
                        param.data += - lrd_mask * lr * m_hat
                else:
                    param.data += - lrd_mask * lr * m_hat / (torch.sqrt(v_hat) + eps)

                if weight_decay:
                    param.data -= weight_decay * param.data

                group['m'][idx] = m
                group['v'][idx] = v

            group['t'] += 1


class RMSProp(Adam):
    """
    RMSprop as proposed by http://www.cs.toronto.edu/~tijmen/csc321/slides/lecture_slides_lec6.pdf.
    Note that this implementation, unlike the original RMSprop, uses bias-corrected moments.
    """
    def __init__(self, params, lr, beta2):
        super(RMSProp, self).__init__(params, lr, beta2=beta2, beta1=0)


class Lookahead(Optimizer):
```

```
    """
    Lookahead Optimization as proposed by https://arxiv.org/abs/1907.08610.
    This is a wrapper class that can be applied to an instantiated optimizer.
    """
    def __init__(self, optimizer, k=5, alpha=0.5):
        self.optimizer = optimizer
        self.k = k
        self.alpha = alpha
        self.param_groups = optimizer.param_groups

        self.counter = 0
        for group in optimizer.param_groups:
            group['phi'] = []
            for param in group['params']:
                phi_param = torch.ones_like(param).mul_(param.data)
                group['phi'].append(phi_param)

    def step(self):
        if self.counter == self.k:
            for group_idx, group in enumerate(self.param_groups):
                for idx, _ in enumerate(group['phi']):
                    theta = self.optimizer.param_groups[group_idx]['params'][idx].data
                    group['phi'][idx] = group['phi'][idx] + self.alpha * (theta - group['phi'][idx])
            self.counter = 0
        else:
            self.counter += 1
            self.optimizer.step()


class GradientNoise(Optimizer):
    """
    Gradient Noise as proposed by https://arxiv.org/abs/1511.06807.
    This is a wrapper class that can be applied to an instantiated optimizer.
    """
    def __init__(self, optimizer, eta=0.3, gamma=0.55):
        self.optimizer = optimizer
        self.eta = eta
        self.gamma = gamma
        self.t = 0
        self.param_groups = optimizer.param_groups

    def step(self):
        normal = torch.empty(1).normal_(mean=0, std=np.sqrt(self.eta/((1+self.t)**self.gamma)))\
            .to(self.optimizer.param_groups[0]['params'][0].device)
        for group_idx, group in enumerate(self.param_groups):
            for idx, param in enumerate(group['params']):
                self.optimizer.param_groups[group_idx]['params'][idx].grad.data += normal
                self.optimizer.step()
                self.t += 1


class GradientDropout(Optimizer):
    """
    Gradient dropout as proposed by https://arxiv.org/abs/1912.00144.
    This is a wrapper class that can be applied to an instantiated optimizer.
    Note that this method does not improve optimization significantly and
    is only here for comparison to Learning Rate Dropout.
    """
    def __init__(self, optimizer, grad_retain=0.9):
        self.optimizer = optimizer
        self.grad_retain = grad_retain
        self.grad_bernoulli = Bernoulli(probs=grad_retain)
        self.param_groups = optimizer.param_groups

    def step(self):
        for group_idx, group in enumerate(self.param_groups):
            for idx, param in enumerate(group['params']):
                grad_mask = self.grad_bernoulli.sample(param.size()).to(param.device)
                self.optimizer.param_groups[group_idx]['params'][idx].grad.data *= grad_mask
                self.optimizer.step()
```